commands

1) Run all tests
```
npx playwright test
```

2) Run a specific test file
```
npx playwright test tests/example.spec.js
```

3) Run in headed mode (browser window visible)
```
npx playwright test --headed
```

4) Run a specific file in headed mode
```
npx playwright test tests/example.spec.js --headed
```

5) Run a test with a specific title
```
npx playwright test -g "title"
```

6) Run only the tests that failed in the last run
```
npx playwright test --last-failed
```

7) Run on a different browser
```
npx playwright test --project=chromium
npx playwright test --project=firefox
npx playwright test --project=webkit
```

8) Run on multiple browsers
```
npx playwright test --project=chromium --project=firefox
```

9) Debug the test in inspector mode (step by step)
```
npx playwright test --debug
npx playwright test tests/example.spec.js --debug
```

10) See the HTML report
```
npx playwright show-report
```

Annotation

-->Annotations in Playwright are special instructions used to provide additional information about a test and control how the test should be executed.

-->They can be used to skip tests, mark expected failures, identify tests that need fixing, increase test timeout, or run specific tests.

-->Playwright provides built-in annotations such as:

test.skip()
test.only()
test.fail()
test.fixme()
test.slow()
test.describe()
test.setTimeout()

1)test.skip()

-->test.skip() is used to skip a test intentinally.
-->Playwright detects the test but does not execute it, and it shows as "skipped" in the report.

```
import {test} from "@playwright/test"
Run | Debug | Inspect
test("test1",()=>{
console.log("test1 executed");
})
test.skip("test2",()=>{
    console.log("test2 executed");
})
Run | Debug | Inspect
test("test3",()=>{
    console.log("test3 executed");
})
```

2)test.only()

-->test.only() is used to execute only the selected test or test group while Playwright skips all the remaining tests during that run.

```
import {test} from "@playwright/test"
Run | Debug | Inspect
test("test1",()=>{
console.log("test1 executed");
})
Run | Debug | Inspect
test("test2",()=>{
    console.log("test2 executed");
})
Run | Debug | Inspect
test.only("test3",()=>{
    console.log("test3 executed");
})
```

3)test.fail()

-->test.fail() is used to mark a test that is expected to fail because of a known bug or unsupported feature.
-->If the test fails, Playwright reports is as passed.
-->If the test passes, Playwright reports is as failed.

-->two things can happen if we mark test as fail:

What happens	Playwright result	Meaning
Test fails	✅ Passed	Your prediction was correct
Test passes	❌ Failed	Your prediction was wrong, so check it

```
test.fail("Annoatation fail",async({page})=>{
    console.log("wrong url");
    await page.goto("https://demowebshop.tricentis.cm/")

})
```

4)test.fixme()

-->test.fixme() is used to mark a test that needs to be fixed because either the test or the functionality it checks is currently broken or unstable.
-->When Playwright encounters test.fixme(), the test is not executed.
-->The test remains in the test suite, but Playwright skips its execution.

Why do we use test.fixme()?
-->The test is currently broken.
-->The application feature is not ready.
-->The test needs changes before it can run correctly.
-->There is a known issue that needs to be fixed.

```
test.fixme("fixme",async({page})=>{
    console.log("wrong url");
    await page.goto("https://demowebshop.tricentiscom/")
})
```

5)test.slow

-->test.slow() is used to mark a test as slow when it is expected to take longer than normal to run/default time. It triples the test's timeout
-->When test.slow() is used, Playwright increases the timeout for that test, allowing it more time to complete.
-->test.slow() does not make the test run slower.
-->It simply tells Playwright:
"This test may take longer, so give it more time before considering it timed out."

```
Run | Debug | Inspect
test("slow",async({page})=>{
    test.slow()
    console.log(test.info().timeout)//print 90 sec
    await page.goto("https://demowebshop.tricentis.com/")
})
```

Easy way to remember:
Default timeout = 30 seconds
With test.slow() = 30 × 3 = 90 seconds

6)test.describe

-->test.describe() is a Playwright method used to group multiple related tests into one logical block (test suite).
-->It helps organize tests that belong to the same feature, module, or functionality.

```
Run | Debug | Inspect
test.describe("login",async()=>{
    Run | Debug | Inspect
    test("valid login feature",async({page})=>{
        console.log("valid login")
    })
    Run | Debug | Inspect
    test("invalid login feature",async({page})=>{
        console.log("invalid login")
    })
})
```

7)test.setTimeout

-->test.setTimeout() is used to set or change the maximum time allowed for a test to complete.
-->If the test takes longer than the specified timeout, Playwright fails the test due to a timeout.

```
test("slow",async({page})=>{
    test.setTimeout(60000);
    await page.goto("https://demowebshop.tricentis.com/")
})
```